

Java + DSA - Table of Contents

Java prerequisites first (the DSA-relevant slice only - revise abstractly, then stop), followed by the DSA topics in sequence.

Part A - Java to Revise Before DSA

1. Syntax & basics

1. Variables, data types, operators
2. Input/output (Scanner, BufferedReader for fast input)
3. Control flow (if/else, for, while, for-each)
4. Methods (parameters, return, overloading)

2. Core building blocks

5. Arrays (1D and 2D)
6. Strings (immutability, common methods) and StringBuilder
7. Recursion basics (call stack, base case)
8. Wrapper classes & autoboxing (Integer, Long), int/long limits & overflow

3. OOP essentials (light - just enough)

9. Classes, objects, constructors
10. Defining custom classes (e.g. ListNode, TreeNode)
11. Comparable and Comparator (for custom sorting)

4. Collections framework (be fluent here - this is the real toolbox)

12. ArrayList, LinkedList
13. HashMap, HashSet
14. TreeMap, TreeSet
15. Stack
16. Queue and Deque (ArrayDeque)
17. PriorityQueue (heaps)
18. Generics (basic usage) and iteration

5. Utilities

19. Sorting: Arrays.sort, Collections.sort, custom comparators (with lambdas)
20. Math utilities, Character methods
21. Big-O (time & space complexity) - concept, learn right before DSA

Part B - DSA Topics in Sequence

1. Basics (complexity, basic math, patterns) + basic recursion + basic hashing
2. Sorting algorithms (bubble, selection, insertion, merge, quick)
3. Arrays (easy -> medium -> hard)
4. Binary Search (on arrays, and 'on answers')
5. Strings (basic -> medium)
6. Linked List (singly, doubly, medium/hard operations)

7. Recursion (subsequences, combinations) + Backtracking
8. Bit Manipulation
9. Stacks & Queues (+ monotonic stack, implementations)
10. Sliding Window & Two Pointer
11. Heaps / Priority Queue
12. Greedy Algorithms
13. Binary Trees (traversals, views, properties)
14. Binary Search Trees
15. Graphs (BFS, DFS, topological sort, shortest paths, MST, disjoint set union)
16. Dynamic Programming (1D -> 2D -> subsequences, strings, stocks, partition DP)
17. Tries
18. Advanced/optional (string algorithms like KMP; segment trees / Fenwick tree)

Notes

Part A is a revision sprint, not a course - a few focused sessions, heaviest on the Collections framework (items 12-17), then move on. The sheet will surface the rest.

In Part B, dynamic programming (16) and graphs (15) are where most people stall - that is expected. Everything before trees is your foundation; build it solid and the back half gets much easier.